Nonogram解題中以邏輯規則進行單行求解效益之探討

林文妍　　黃國展
臺中教育大學資訊工程學系
clps10070623@gmail.com　　kchuang@mail.ntcu.edu.tw

摘要

Nonogram 是一種益智遊戲，同時也是國內外電腦對局競賽中的比賽項目之一。單行(一橫列或一直行)是 Nonogram 解題的基礎單位，每一個單行都有屬於自己的線索資訊。透過一單行本身的線索資訊及解題過程中由其他單行部分解所不斷增加之限制條件，可以逐步求解出該單行所有格子的答案。近年來文獻及實際競賽的程式中在單行求解部分，大都採用所謂嘗試所有組合(combinatorial)的求解方法，主要是能善用電腦的快速計算能力。而在較早期的文獻中曾出現過以邏輯規則來進行單行求解的探討，這種求解方式較貼近人類本身解題時的思維模式。從不同的層面來看，這兩種不同的單行求解方法可說各擅勝場，因為嘗試所有組合的求解方法不會遺漏任何的可能性，所以可以找出較多格子的解答，但是利用邏輯規則求解由於在條件成立時可以直指核心，不必多做額外的嘗試，因此在求解效率上具有一定的優勢。我們在本篇論文中詳細分析單行邏輯規則求解方法的相對優劣勢，並嘗試發展新的邏輯規則與整合性方法。研究結果除了顯示以邏輯規則進行 Nonogram 中單行求解極具效率上的潛力(可達數倍差距)之外，也不失為一個研究人工智慧領域中邏輯規則發掘與推演等相關主題的良好媒介。

關鍵詞： Nonogram、單行求解、邏輯規則推演、嘗試所有組合求解

1. 前言

Nonogram 是一種邏輯益智遊戲，其玩法如圖 1 所示：左邊為遊戲初始狀態，以一個空白的方形網格為基底，每個橫列左方及每個直行上方都有一組數字，這些數字在此稱為線索，代表該行或是該列中所應塗黑的格子數及其分佈條件。以圖 1 中的第二列為例，其左方數組為「2, 1」，代表該列應連續塗黑兩個格子及單獨塗黑一個格子，且這兩個黑色區塊間應以至少一個以上的白色格子作為區隔。在所有行列內容皆符合其線索後，便稱之為得到該題 Nonogram 的正確解答之一(因為一題 Nonogram 可能會有多組同時滿足行列線索的不同解答)。

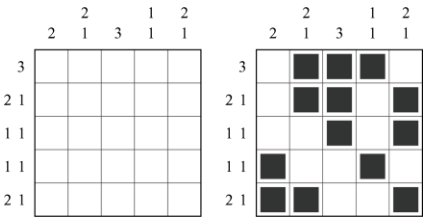

圖 1: 5×5 Nonogram 實例; (左)初始狀態; (右)其解

一般而言，Nonogram 題目的難度會隨著行列數目增加而提高，一些提供 Nonogram 遊戲題目的網站上(如 https://www.puzzle-nonograms.com/)通常會提供不同大小的題目供玩家嘗試，例如前述網站便提供了 5×5、10×10、15×15、20×20、25×25 等五種不同大小的 Nonogram 謎題。目前國內外主要 Nonogram 相關競賽(如 TCGA, TAAI, ICGA Computer Olympiad)都是採用 25×25 的 Nonogram 題目，這樣大小的題目人類本身解來已頗感吃力，適合在競賽中由實作不同解題演算法的電腦程式來進行求解。比賽時不同程式依序(不得跳題)求解相同的 1000 題 25×25 由易至難的 Nonogram 謎題，以所解出的題數越多或是所花的時間越少(當求得解答的題數相同時)，來判定勝負。

Nonogram 反覆求解的過程中，通常每次以一個單行(一橫列或一直行)為單位，透過其本身的線索資訊及其他單行當時已得之部分解所加諸的新限制條件，來進一步求解出該單行中某些尚未填上顏色格子的答案。在有些如[3]中的解題演算法裡，單行求解甚至會不斷地被後續階段呼叫，所以其在 Nonogram 解題演算法研究中占據重要的一環。對任何謎題而言，只要能求得正確解答，並不侷限只能用何種特定方法。但人類本身擅長邏輯思考，因此在求解 Nonogram 謎題時，透過觀察，然後從邏輯規則的推理角度切入，是一個很自然的作法，可能也是當初發展出此類益智遊戲的目的之一。所以在早期 Nonogram 解題研究的相關文獻中，的確也曾出現過以邏輯規則為基礎的單行求解方法[1][2]。

以邏輯規則求解的優點是，一旦發現目前某單行的內容符合特定的樣式，可以適用某些先前所歸納出與驗證過之邏輯規則的話，就可以直接決定某些原本尚未塗上顏色格子的答案，極具效率；但缺點則是邏輯規則可能變得十分複雜，並且由於已知的邏輯規則為根據人類經驗所逐漸歸納出，並不確定具有完備性，因此若單純靠邏輯規則求解，實際上可能會導致某些格子的解答永遠無法確定的困境，使得無法解出某些 Nonogram 謎題的答案。為了解決此一問題，後續的 Nonogram 解題研究逐漸走向所謂嘗試所有組合(combinatorial)的單行求解方法[3][4][5][6][7]。

嘗試所有組合單行求解方法理論上具有完備性，不會錯失任何可能性，可以找出當時條件下(包括單行本身的線索資訊及解題過程中不斷增加之其他單行部分解所加諸的限制條件)所有可以被確定答案的格子顏色，問題只在所需花費的時間而已。

由於具有此一優勢，並且可以善用電腦的快速計算能力，因此近年來文獻及實際競賽的程式中在單行求解部分，大都採用此一作法，並透過不斷研究出新的高效率組合搜尋方法(如[3]中的動態規劃方法)及程式實作優化技巧，使得各參賽解題程式的速度每年都有持續的進步。當然，採用嘗試所有組合求解的方法雖然有機會比單純使用邏輯規則在單行求解階段找出更多格子的解答，但通常仍無法解出整題 Nonogram 的答案，後續階段往往需要搭配如回溯搜尋(backtracking)等方法來進行。

嘗試所有組合的單行求解方法已經偏離人類本身擅長的邏輯思維模式，而轉向電腦較具優勢的計算導向搜尋求解模式。從不同的層面來看，這兩種單行求解方法可能是各擅勝場的。以直觀的想法來說，雖然嘗試所有組合的方法可能可以填出較多格子，但以每多填出一格所需花費的時間來衡量的話，以邏輯規則進行單行求解的填格效率極有可能是比較高的。因此，本篇論文將聚焦於以邏輯規則進行單行求解，並嘗試詳細比較這兩種方法的解題效率，進而提出新的改良與整合性 Nonogram 單行求解方法，期望能進一步有效提升整體的 Nonogram 解題速度。

2. 嘗試所有組合單行求解方法

許多 Nonogram 解題相關研究在單行求解的部分都是採用嘗試各種組合的求解作法。這類單行求解方法通常可進一步分成兩個部分，首先是能否有解的判斷(即 fixability 問題[3][6])，進而才以此為基礎嘗試進行單行中各格的求解(即[3]中的 paint)。

例如 Batenburg 與 Koster 在[6]中便以遞迴運算模式來進行嘗試所有組合的單行求解。他們所提方法的特性是用如底下的單一 *Fix* 遞迴函式定義來處理 fixability 問題。每一單行 fixability 問題的判斷過程都可視為從該行的最右邊一格與最後一個線索開始，因此函式定義中的 i 表示剩下尚未檢查的格子個數，而 j 則從最後一個線索條件的編號開始遞減。與圖 1 中行列線索不同之處在於此 *Fix* 函式中的線索 j 除了包含連續塗黑的區塊外，也納進了連續留白的區塊，因此 j 的個數約為圖 1 中每行線索數量的兩倍。另外，與已經被指定長度之連續塗黑區塊不同之處在於連續留白區塊的長度是不固定的，在線索中除了限制其長度至少為 1 之外，並無其他條件[6][10]。

$$Fix(i,j) = \bigvee_{p=\max(i-b_j, A_{j-1}, L_i^{a_j}(s))}^{\min(i-a_j, B_{j-1})} Fix(p, j-1). \tag{1}$$

基於類似的單行求解概念，交通大學資工系吳毅成教授等人在[3]中則進一步提出了新的改良方法。在他們新的嘗試所有組合單行求解方法中，將 fixability 問題的判斷模式改成由如底下的 *Fix*, *Fix1*, *Fix0* 等三個函式來共同完成。其中 *Fix1* 就是根據 Nonogram 題目所提供的線索，由右至左做填 1(塗黑)的動作，而 *Fix0* 則是進行補 0(留白)的動作。跟前述 Batenburg 與 Koster 等人的方法比起來，這個新方法的主要差異在於函式定義中的線索部分(即 j)已經不包含連續留白區塊，這一部分改由 *Fix0* 函式來處理。這樣的轉變除了讓線索部分與 Nonogram 問題中的原始線索資訊更為一致之外，也讓整個 fixability 問題的判斷過程更為清楚，也更容易理解與實作，此為其優點。

$$\mathrm{Fix}(i,j) = \begin{cases} \text{true}, & \text{if } i=0 \text{ and } j=0 \\ \text{false}, & \text{if } i=0 \text{ and } j \geq 1 \\ \mathrm{Fix0}(i,j) \vee \mathrm{Fix1}(i,j), & \text{otherwise} \end{cases}$$

$$\mathrm{Fix0}(i,j) = \begin{cases} \mathrm{Fix}(i-1,j), & \text{if } s_i \in \{0,u\} \\ \text{false}, & \text{otherwise} \end{cases}$$

$$\mathrm{Fix1}(i,j) = \begin{cases} \mathrm{Fix}(i-d_j-1, j-1), & \text{if } j \geq 1, i \geq d_j+1, \text{ and } s_{i-d_j} \ldots s_i \text{ matches } \sigma(d_j) \\ \text{false}, & \text{otherwise.} \end{cases}$$

在能夠處理 fixability 問題後，[3]中的單行求解方法便以底下的 *Paint* 遞迴函式定義，在嘗試所有組合可能後，為現階段已能確定其解的格子填上答案。其判斷方式為：若是 *Fix0* 及 *Fix1* 函式只有其一成立，則填入以其所代表的相對應值為基礎所進行的遞迴延伸結果；若皆不成立，則此單行在目前的條件下無解；若皆能成立，並不是代表填 0 或填 1 都可以，而是必須針對 *Fix0* 及 *Fix1* 這兩種可能的遞迴延伸結果進行衝突檢查，方能判斷其中有哪些格子是能在現階段求得解答的。

$$\mathrm{Paint}(i,j) = \begin{cases} \phi, & \text{if } i=0 \\ \mathrm{Paint}'(i,j), & \text{otherwise} \end{cases}$$

$$\mathrm{Paint}'(i,j) = \begin{cases} \mathrm{Paint0}(i,j), & \text{if } \mathrm{Fix0}(i,j) \text{ and not } \mathrm{Fix1}(i,j) \\ \mathrm{Paint1}(i,j), & \text{if not } \mathrm{Fix0}(i,j) \text{ and } \mathrm{Fix1}(i,j) \\ \mathrm{Merge}(\mathrm{Paint0}(i,j), \mathrm{Paint1}(i,j)), & \text{otherwise} \end{cases}$$

$$\mathrm{Paint0}(i,j) = \mathrm{Paint}(i-1,j)0$$

$$\mathrm{Paint1}(i,j) = \mathrm{Paint}(i-d_j-1, j-1)\sigma(d_j)$$

$$\mathrm{Merge}(s_1 s_2 \ldots s_i, t_1 t_2 \ldots t_i) = m_1 m_2 \ldots m_i, \quad \text{where } m_k = \mathrm{Merge}C(s_k, t_k)$$

$$\mathrm{Merge}C(s_k, t_k) = \begin{cases} 0, & \text{if } s_k = t_k = 0 \\ 1, & \text{if } s_k = t_k = 1 \\ u, & \text{otherwise.} \end{cases}$$

前述的遞迴函式在計算過程中難免會出現帶相同參數之重複函式呼叫，導致不必要的重複計算。這個問題可在程式實作時引進動態規劃機制(dynamic programming)[11]來加以避免，提升計算效率，在[3][6]當中也都有提到如此的作法。

3. 以邏輯規則進行單行求解

我們在本篇論文中，首先以[1]中所提出的一個前置處理(preprocessing)規則及五個邏輯規則為基礎，開始導入邏輯規則單行求解方法。其中前置規則是應用在 Nonogram 解題過程剛開始的空白單行上，而後續五個邏輯規則可以持續應用在過程中已有部分解的單行上。這些邏輯規則皆利用如圖 2 所示的單行極左可能解答(Left-most solution)和極右可能解答(Right-most solution)這兩個概念，從中觀

察出可以確認答案的格子。在這裡，我們使用■作為塗黑的格子，×代表空白的格子，以及 表示仍不確定該塗黑或留白的未知格子。

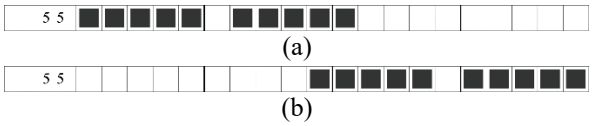

圖 2: 單行極左(a)和極右(b)可能解答示意

3.1 前置處理規則

前置處理規則主要是整理並獨立出[8]中三個如下能簡單描述或以數學公式進行闡述的規則。

- 單一線索完全填滿(complete fill)：當該單行的線索只有一個，且須塗黑的格子數與 Nonogram 題目的單行格子數相等時，即可得出唯一解，所有格子皆須塗黑，圖 3 為 20×20 Nonogram 中一個符合此情況的例子。

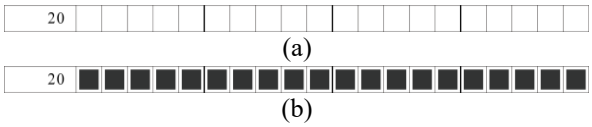

圖 3: 單一線索完全填滿例子

- 多重線索完全填滿(Complete fill with definite blanks)：假設線索的數組內容為 n_i，並總共有 k 個數字，在符合下列情況時，則此規則的條件成立，其中 N 為該單行長度。圖 4 為一實例。

$$\sum_i n_i + (k-1) = N$$

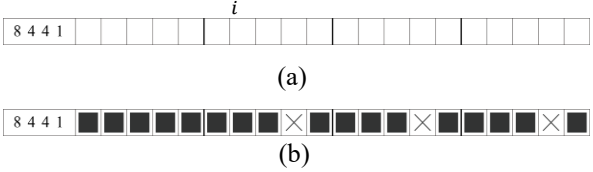

圖 4: 多重線索完全填滿實例

- 部分填滿(Partial fills)：以圖 5 為例，該單行的線索為「15」，則可以觀察到中間有 10 個格子在所有可能的解答中都會被塗黑，因此便可以提前確定那 10 個格子的答案，亦即若當 $l = M - (\sum_i n_i + (k-1)$ 時 $n_i - l > 0$ 成立的話，則能確定其中 $n_i - l$ 個格子必須塗黑。

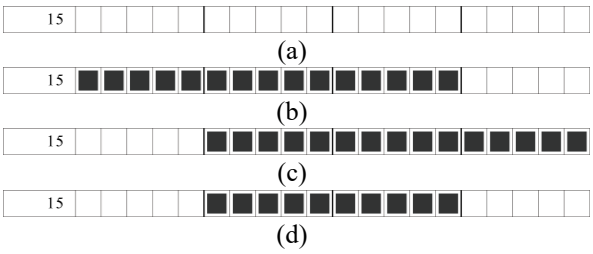

圖 5: 部分填滿的例子

3.2 規則一

以已經求得部分解的單行中確定必須留白的格子來劃分出數個未知區塊(如圖 6 的實例中，用一個必須留白格子劃分出兩個未知區塊)，若發現某些線索僅能填入特定的未知區塊中(如圖 6 中的第一個線索無法填入最左邊的未知區塊中)，我們就能利用每個線索活動範圍被侷限的特性來進行極左可能(圖 6(b))與極右可能解答(圖 6(c))的交叉比對，兩個可能解答中各個線索的重疊處即為能被確定答案的格子(圖 6(d))。

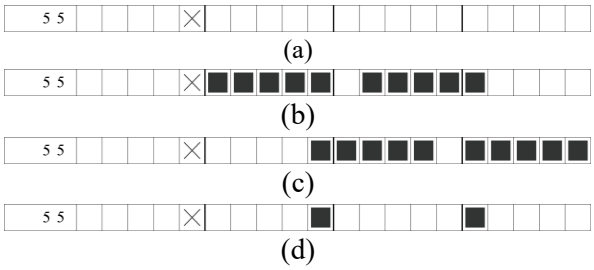

圖 6: 規則一實例

3.3 規則二

以單行中已被塗黑的格子為限制條件(圖 7(a))，可以進行極左可能(圖 7(b))與極右可能解答(圖 7(c))位置的確認，將兩個可能解答進行交叉比對，即可以判斷出某些必需留白的格子(圖 7(d))。

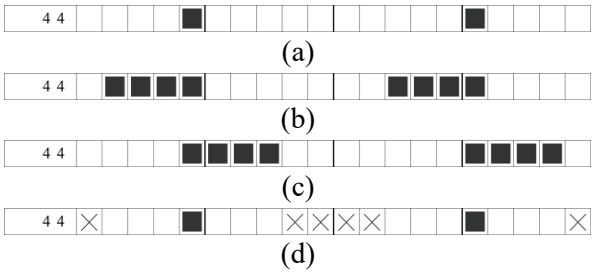

圖 7: 規則二實例

3.4 規則三

以單行已知部分解中某一相鄰的留白與塗黑區塊為基準(圖 8(a))，進行極左可能(圖 8(b))與極右可能解答(圖 8(c))位置的確認，這兩個位置同時也將限縮住所有可能解答的活動範圍，因此藉由取所有可能解答的交集，可以判斷出某些格子的正確解答(圖 8(d))。

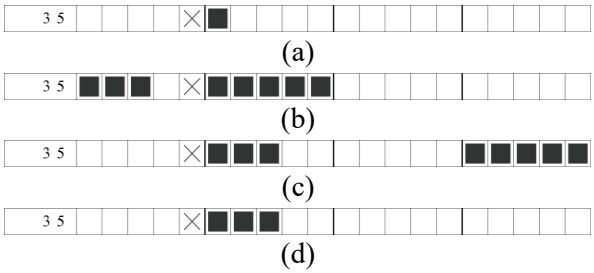

圖 8: 規則三實例

3.5 規則四

如下圖 9 所示，在單行已有部分解的情況下，某些線索的活動範圍將被侷限，在確認極左可能(圖 9(b))與極右可能解答(圖 9(c))的位置後，如果發現某些未知格子區塊沒有足夠空間可以容納任何線索的話，即可確認某些格子必須留白(圖 9(d))。

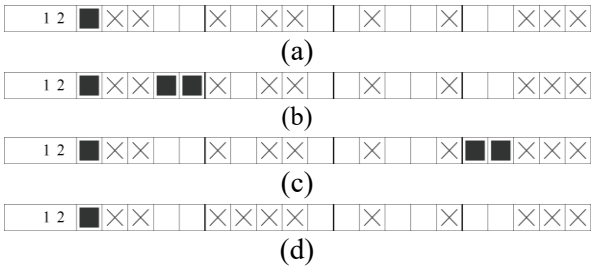

圖 9: 規則四實例

3.6 規則五

以下圖 10 為例，在單行部分解中某些塗黑的格子(圖 10(a))，在確認極左可能(圖 10(b))與極右可能解答(圖 10(c))的位置後，可以有效限制住其他所有可能解答的活動範圍，因而可以進一步判斷出某些不能塗黑的格子，即可得知其答案為留白(圖 10(d)中塗黑格子的左側格子)。

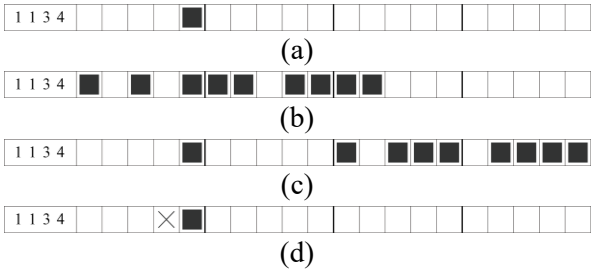

圖 10: 規則五實例

4. 延伸及新增邏輯規則

由於先前嘗試以邏輯規則進行 Nonogram 求解的相關研究[1][2]，乃是分別提出，彼此間並未做過對照比較，因此我們在本篇論文中以[1]中的五個邏輯規則為基礎，除了評估每個邏輯規則的實際效益外，亦嘗試改良整合這些邏輯規則以提升解題效率。單行求解是 Nonogam 解題的基礎，並且其計算效率相較於許多後續階段的方法，如窮舉搜尋或回溯搜尋(backtracking)等[3]，還要快上許多，若無法盡量在此階段求出較多格子的解，以達到足夠的限縮後續搜尋空間效果的話，將會降低整體的解題速度。此外，在有些 Nonogram 解題方法中[3]，單行求解過程更是會不斷在後續的回溯搜尋等階段中重複被運用到，更突顯出其重要性。

4.1 規則整合

[1][2]中的邏輯規則乃一前一後分別提出([1]在前，而[2]在後)，但並未進行相互比較或引用，雖然[2]中的整體邏輯規則數量較[1]為多，但經過分析，我們發現有許多邏輯規則是與[1]中相似或具有相同效果的。不過[2]中的 Rule 1.5 似乎是一個具有潛力的新規則，因此我們把它整合進本篇論文的以邏輯規則進行單行求解架構中。

此規則可以第三節中相同的方式加以描述如下: 當單行部分解中有被未知解區塊隔開之空白格子與黑色格子，且其距離小於所有可能置於其上線索之長度時圖(11(a))，將有機會透過取極左可能解(圖 11(b))與極右可能解((圖 11(c))之交集來求得部分格子的解答。以圖 11 為例，圖 11(a)中已知部分解裡有一空白格子與塗黑格子，其距離為 2，小於「3,4」這兩個線索的長度，則取最小的線索「3」與 2 相減，得出可往原部分解中塗黑格子的右邊再多塗黑 1 格，如圖 11(d)所示。

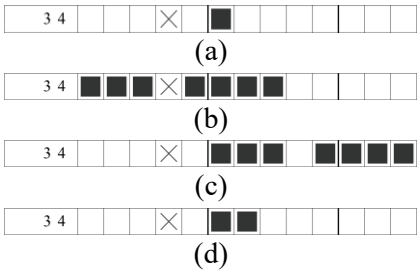

圖 11: [2]中 Rule 1.5 實例

4.2 規則延伸

底下圖 12 描述一個我們根據[1]中既有的邏輯規則所延伸出來的新規則應用。如圖 12 所示，圖 12(a)中的部分解含有相鄰的留白格子及塗黑格子，此種型態代表其為某一個線索塗黑區塊的起點。因此，藉由極左可能解(圖 12(b))與極右可能解(圖 12(c))位置的確認，可以發現有一個格子不可能被塗黑，因此求得其解答為留白(圖 12(d))。此延伸規則整合了[1]中規則三的初始部分解狀態以及規則五的求解判斷方法。

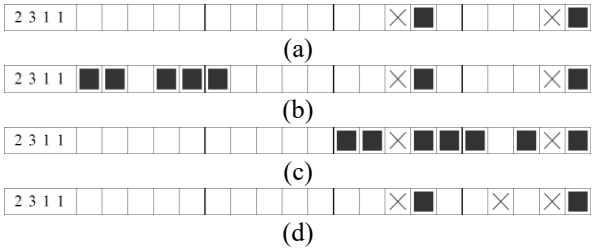

圖 12: 延伸規則實例

5. 兩種單行求解方法實驗分析

為了進行不同單行求解方法的實驗分析，我們將單行求解的程式從整體 Nonogram 解題流程中獨立出來。在以邏輯規則進行單行求解的部分，其執行架構如下所示[1]。

Procedure 邏輯規則單行求解 (*G*)

1. 執行 3.1 節前置處理
2. **while** (前一輪仍有進展)

3. **foreach**(前一輪有新增填點的單行)
4. 求得該單行之極左與極右可能解答
5. 執行 3.2 節邏輯規則
6. 執行 3.3 節邏輯規則
7. 執行 3.4 節邏輯規則.
8. 執行 3.5 節邏輯規則
9. 執行 3.6 節邏輯規則
10. 執行 4.1 節邏輯規則
11. 執行 4.2 節邏輯規則
12. **end foreach**
13. **end while**

End procedure

前述的執行架構在整體 Nonogram 解題中所扮演的角色，約略等於[3]中如下的 Propagate 階段，是整體 Nonogram 解題流程中的第一階段，除了很簡單的題目之外，通常在此階段尚不能解完整題 Nonogram。

Procedure PROPAGATE (G)

1. $\Pi(G) \leftarrow ø$; // $\Pi(G)$ is a set of updated pixels
2. *Put all rows and columns of G into* $\mathcal{L}(G)$
3. **while** ($\mathcal{L}(G) \neq \emptyset$) **do**
4. *Retrieve one line L from* $\mathcal{L}(G)$
5. *Let the line be* S_i^R *//row i;*
6. **if** (notFix(S_i^R, D_i^R)) **then** status(G) ← CONFLICT;
   return
7. $S_i^R \leftarrow Paint(S_i^R, D_i^R)$
8. *Let* Π *be the newly painted pixels of* S_i^R
9. *For each pixel in* Π, *put its column into the list* $\mathcal{L}(G)$*, if not in* $\mathcal{L}(G)$ *yet*
10. $\Pi(G) \leftarrow \Pi(G) \cup \Pi$; // collect all painted cells in this
11. **end while**
12. **if** (*all pixels are painted*) **then** status(G) ← SOLVED
13. **else** status(G) ← INCOMPLETE

End procedure

底下的實驗除了比較嘗試所有組合[3]及基於邏輯規則來進行單行求解這兩類方法之外，也分析了不同邏輯規則的求解效益。由於我們目前的邏輯規則單行求解程式是初步的實作結果，尚未進行優化，為了能進行客觀的演算法層次實驗比較，以及看出實作優化對整體解題效率的影響，我們在實驗中總共比較了兩個不同版本的嘗試所有組合單行求解程式，其中一個是我們為了此次實驗，根據[3]中的方法所實作出來的版本，其實作上的技巧水平大約等同於我們的邏輯規則單行求解程式，兩者也都還未經程式實作上的優化，而另一個版本雖然當初也是以[3]中的方法為基礎，但已歷經學長姐們數年來的進一步改良與實作優化[20]，具有極佳的解題效率，也曾獲得過幾屆的 TCGA Nonogram 競賽金牌。

我們以過去三年 TCGA 競賽題目(2019~2021，每年有 1000 題)進行實驗，來比較不同的單行求解方法。實驗結果中不同方法或邏輯規則間的比較基準包括了整題 Nonogram 在此階段所能夠求得解的總格子數、所需總時間、單格平均求解時間等三個指標。所有解題程式皆用 C++撰寫，實驗平台為安裝 Windows 11 64 Bits 的筆記型電腦，其中處理器為 Intel(R) Core(TM) i7-7700HQ CPU @ 2.80GHz、記憶體為 16GB、GCC 編譯器版本則為 TDM-GCC 4.9.2 64-bit。

5.1 兩種單行求解方法比較

底下表 1、2、3 分別為三年不同競賽題目的實驗結果，其中填格數為平均每題在單行求解階段所可以找出解答的格子數目(相較於一題 25×25 Nonogram 的總格數，在此第一階段即能解出答案的格子僅為少數)，總時間為找出這些格子解答所花的全部時間，而單格時間則是換算下來平均求得一格答案所需的時間。

表 1: TCGA 2019 (不同單行求解方法)

	組合(未優化)	組合(優化)	邏輯(未優化)
填格數	31.547	31.547	30.904
總時間(秒)	0.497117	0.002172	0.000316
單格時間(秒)	0.015758	0.000069	0.000010

表 2: TCGA 2020 (不同單行求解方法)

	組合(未優化)	組合(優化)	邏輯(未優化)
填格數	28.564	28.564	27.937
總時間(秒)	0.492603	0.002119	0.000418
單格時間(秒)	0.017246	0.000074	0.000015

表 3: TCGA 2021 (不同單行求解方法)

	組合(未優化)	組合(優化)	邏輯(未優化)
填格數	33.283	33.283	32.894
總時間(秒)	0.497246	0.002016	0.000476
單格時間(秒)	0.014940	0.000061	0.000014

從三張表的實驗結果可以看出一致性的趨勢，以邏輯規則進行單行求解的作法在效率上有明顯的優勢，若與同樣未經程式實作優化的嘗試所有組合版本(第二欄)比較的話，平均單格求解速度的提升可達數百倍，顯示出其在演算法層次上的極大優勢，即使是跟經過數年

實作優化的嘗試所有組合版本(第三欄)比較，也仍有數倍的提升。從第一、二欄的數據比較亦可發現程式實作優化(包括動態規劃等機制)對求解效率有極大的影響(可達數十倍)，這也是邏輯規則求解程式未來需要持續精進的一個項目。另外，雖然以邏輯規則進行單行求解的方法具有平均單格求解效率的優勢，但在此階段所能求得解答的總格子數方面卻略遜一籌，平均少了近一格，雖然差距不大，仍是後續努力的方向之一，以避免將過多未求得解的格子留至回溯搜尋階段，影響整體解題速度。

5.2 不同邏輯規則間的比較

由於我們的邏輯規則單行求解架構含前置處理共有八個不同的規則，因此底下的實驗嘗試比較不同邏輯規則的求解效率。我們以兩個不同版本的邏輯規則求解程式來進行底下的實驗，其中一個版本僅包括[1]中所提出的前置規則與五個邏輯規則(表中第二欄)，而另一個版本則加上了 4.1 及 4.2 節中所描述的整合及延伸規則(第三欄)。因為在單行求解過程中，有的格子可能可被一個以上的邏輯規則解出，而邏輯規則的套用有一定的順序，因此在邏輯規則數量及順序不完全相同的情況下，同一個邏輯規則在兩個不同版本裡的填格數可能會有所不同。

表中由上到下的順序即是各邏輯規則在單行求解過程中被套用的順序。前置規則永遠是第一順位，而規則二、四、五在盤面狀態較為複雜時容易同時發生，難以界定究竟是屬於哪一個規則，因此我們將它們整合成一個群組進行套用。規則一及三需仰賴留白格子的前置狀態，因此我們將它們移至負責確認留白的規則二、四、五之後以提升填格效率。Rule 1.5[2]的判斷機制與規則一相似，因此將它們放在相鄰的順位。

從底下三張表中總填格數那列數據可以看出，我們所整合延伸出的新邏輯規則求解架構的確可以比[1]中原本的邏輯規則多找出一些格子的解答。在表中規則二、四、五因為整合在一起的關係，其總填格數是最高的，但若將其除以三取平均再重新檢視的話，會發現規則一的填格數最高。規則三的填格數最低，除了因為其成立的限制條件較多之外，所需的線索長度要求也較高，亦是原因之一。另外，我們也可以發現 4.1 及 4.2 節中新加入的邏輯規則的確能有效提升總填格數，同時也大幅瓜分了規則一的填格數，並影響其他規則的填格結果，這是因為在判斷上，符合 Rule 1.5[2]的前置狀態容易與規則一的前置狀態混合出現，因此有可能將符合規則一的情況納為己方；而規則延伸的判斷機制則因為與規則五相仿，因此同樣也有機會將規則五的可填格子納為己方，又因為這兩個新增邏輯規則都有利於規則四的進行，因此在有時互補有時競爭的情況下，規則二、四、五的填格數可能出現有時更多(如表 5、6)，而有時反而減少的情況(如表 4)。

表 4: TCGA2019 (不同邏輯規則)

	[1]中邏輯規則	擴充邏輯規則
總填格數	30877	30904
前置處理	8962	8962
規則二、四、五	11727	11722
Rule 1.5 [2]		3400
規則一	10078	6681
規則三	110	111
規則延伸		28

表 5: TCGA2020 (不同邏輯規則)

	[1]中邏輯規則	擴充邏輯規則
總填格數	27831	27937
前置處理	8888	8888
規則二、四、五	10131	10167
Rule 1.5 [2]		2915
規則一	8696	5823
規則三	116	118
規則延伸		26

表 6: TCGA2021 (不同邏輯規則)

	[1]中邏輯規則	擴充邏輯規則
總填格數	32548	32894
前置處理	9243	9243
規則二、四、五	12349	12518
Rule 1.5 [2]		3796
規則一	10853	7212
規則三	103	104
規則延伸		21

從上述實驗結果及分析來看，不同邏輯規則之間有著緊密且難以分割的關係，無論是判斷的機制還是單行的前置狀態，在盤面情況越複雜時更有可能同時符合數個邏輯規則，因此若是能找到規律將相似的規則合併，如目前的規則二、四、五，勢必能有效提升以邏輯規則進行單行求解的效率。

6. 整合性單行求解方法

根據前一節中的實驗結果，我們可以看到以邏輯規則進行單行求解雖然在可求出解答格子部分的平均效率上大幅優於嘗試所有組合的單行求解方法，但在平均每題可求得解答的總格子數方面卻是以接近一格的差距，略遜一籌。於是在此節中，我們嘗試以先進行邏輯規則單行求解(Procedure 邏輯規則單行求解 (G))，再接續以嘗試所有組合求解(Procedure PROPAGATE (G))的方式來整合這兩種方法，其目的是看先進行的邏輯規則求解，能否達到有效限縮後續嘗試所有組合方法所需搜尋空間的效果。如若可能，則有機會達成花費較少時間，就能夠與原本的嘗試所有組合方法求出相同數目格子解答的兩全其美結果。我們將以邏輯規則進行

單行求解的程式分別與前述兩個版本之嘗試所有組合程式進行整合，底下是同樣以過去三年 TCGA 競賽題目所進行的實驗結果。

從實驗結果可以看出若將邏輯規則求解與未經優化的嘗試組合求解版本以先後順序進行整合(第四欄)，的確可以達到預期中的目的，即達到相同求得解之格子總數下，比起第二欄中的數據，仍有較佳的解題效率(雖然效率比起前一節中的單獨作法降低不少)；但若是與優化後的嘗試組合版本整合的話(第五欄)，則魚與熊掌未能得兼，雖然解出了相同數目的格子，但求解速度卻略遜於單獨的嘗試所有組合優化版本(第三欄)。

表 7: TCGA2019 (整合性方法)

	組合(未優化)	組合(優化)	邏輯+組合(未優化)	邏輯+組合(優化)
總時間(秒)	0.497117	0.002172	0.487327	0.002243
單格時間(秒)	0.015758	0.000069	0.015448	0.000071

表 8: TCGA2020 (整合性方法)

	組合(未優化)	組合(優化)	邏輯+組合(未優化)	邏輯+組合(優化)
總時間(秒)	0.492603	0.002119	0.476586	0.002192
單格時間(秒)	0.017246	0.000074	0.016685	0.000077

表 9: TCGA2021 (整合性方法)

	組合(未優化)	組合(優化)	邏輯+組合(未優化)	邏輯+組合(優化)
總時間(秒)	0.497246	0.002016	0.475539	0.002491
單格時間(秒)	0.014940	0.000061	0.014288	0.000075

這樣的結果顯示出目前初步的一前一後整合方式仍嫌粗糙，等於執行了兩輪單行求解過程，額外增加的負擔太大，而邏輯規則求解先行所能達成的限縮嘗試所有組合搜尋空間的效果相當有限，導致只有在與原本求解效率即能領先達數百倍的未經優化嘗試所有組合版本整合時才有機會達到預期效果，而當與原本效率就極高的優化後嘗試所有組合版本(求解效率僅落後邏輯規則求解方法數倍)進行整合時，因為額外增加的負擔超過了限縮搜尋空間的效益，使得整體求解效率輸給單獨的優化後嘗試所有組合版本。因此，後續應朝更細緻緊密的整合方向努力，以盡可能降低整合所帶來的額外負擔，或是往研究出新的邏輯規則，讓單獨邏輯規則求解方法就能填出跟嘗試所有組合求解作法相同數目格子的方向努力。

7. 結論及未來研究方向

以邏輯規則進行單行求解是文獻中曾經出現過的方法[1][2]，但尚未與近年來主流的嘗試所有組合求解方法進行過詳細的分析比較，並且較缺乏後續的相關研究。本篇論文旨在探討 Nonogram 解題過程中，以邏輯規則進行單行求解之效益。我們從[1]中所提出的五個獨立邏輯規則開始，逐步嘗試彼此間的整合及納進其它有增益效果的規則(如[2]中的 rule 1.5)，並進一步發展出新的延伸規則，最後整個邏輯單行求解架構中共包括了一個前置處理規則和七個邏輯規則。

以過去三年 TCGA 競賽題目(每年 1000 題)進行實驗的結果顯示，跟傳統嘗試所有組合[3]的方法比起來，以邏輯規則進行單行求解雖然平均起來每題大約會少求出近一格的答案(未求得解的格子會留待後續回溯搜尋階段求出答案)，但在所需求解時間方面卻有相當大幅度的縮減，比起實作上未優化過的嘗試所有組合版本，單行求解速度快超過千倍，即使跟極度優化的版本比起來，也有數倍的提升(這裡也可看出動態規劃及其他實作優化機制對嘗試所有組合求解方法的重要性)。這樣的實驗結果突顯出以邏輯規則進行單行求解在效益上具有不小的潛力，值得後續更進一步的研究探討。

從目前初步的實驗結果看來，後續研究最優先的目標應該是發掘新的邏輯規則，以彌平目前跟嘗試所有組合求解方法間平均每題約略少求出一格解的差距。我們在本篇論文中參與實驗比較的邏輯規則單行求解程式是初步開發出來的版本，尚未經過實作上的優化與改良。因此，第二個後續待努力的研究方向是邏輯規則單行求解在實作層面的優化，以補強其相對於目前嘗試所有組合版參賽程式已歷經多年實作層面優化上的落差。其他具有潛力，值得嘗試的後續研究方向還包括更進一步緊密整合邏輯求解與嘗試所有組合這兩種方法(利用邏輯規則來有效縮減需嘗試的可能範圍)，以及跨行的邏輯規則發掘等。

Nonogram 競賽主要比的是解題速度，但從學術研究的角度來看，速度並不是唯一的有意義追求目標。以邏輯規則來進行 Nonogram 求解，除了較貼近人類本身的思維模式，具有教育與益智性娛樂

的潛在應用與意義之外，邏輯推演與規則發掘原本也就是人工智慧領域中重要的相關研究主題[12][13][14][15]。Nonogram 解題可以做為進行相關領域研究的一個有利媒介，值得後續努力。

這些邏輯規則及其推演通常會牽涉到因果關聯與推論[11][12][16][17][18]。其實在[3]的 Nonogram 解題架構中，雖然在單行求解階段是採用嘗試所有組合的作法，但在其全面探測(Fully Probingt)階段已有嘗試運用一些邏輯推演的法則，如逆否命題(contrapositive)等，足見其潛力。在能夠進行邏輯規則的發掘與推演之前，發展出一個一致且有效(包括效用與效率)的邏輯規則表達方式，以及 Nonogram 盤面中不同格子間的邏輯關聯描述模型，也是很重要的研究方向之一。

[11][12]中的有向因果圖(Directed Graph of Cause and Effect)是個有潛力的參考架構。邏輯因果圖的建立也有助於在解題過程中適時地將處理標的從 Nonogram 盤面轉移至因果圖上，有機會用來嘗試發掘出目前尚未被觸及之跨單行間的因果邏輯規則。當重點放在因果圖運算時，傳統圖論上的許多演算法也都可望能在推演過程中發揮積極助益的功能。更進一步而言，以邏輯因果圖為媒介，最近頗受矚目的圖神經網路(Graph Neural Networks)[19]相關技術，也就成為一個潛在可能的探索方向。

誌謝

本論文為科技部計畫(MOST 108-2221-E-142-004 -)成果之延伸研究。感謝葉家郡、宋得榮、林芃儀提供程式參與實驗比較。

參考文獻

[1] S. Salcedo-Sanz, A. M. Perez-Bellido, A. Portilla-Figueras, X. Yao, “Solving Japanese Puzzles with Heuristics”, *Proceedings of IEEE Symposium on Computational Intelligence and Games*, 2007.

[2] C. Yu, H. Lee, and L. Chen, "An Efficient Algorithm for Solving Nonograms", *Applied Intelligence*, vol. 35, pp. 18–31, 2011.

[3] I.C. Wu et al., “An Efficient Approach to Solving Nonograms,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 5, no. 3, pp. 251-264, 2013.

[4] Y.C. Chen, and S.S. Lin, “A Fast Nonogram Solver That Won the TAAI 2017 and ICGA 2018 Tournaments,” *ICGA Journal*, vol. 41, no. 1, pp. 2-14, 2019.

[5] D. Berenda, D. Pomeranzb, R. Rabani, B.Raziel, “Nonograms: Combinatorial Questions and Algorithms”, *Discrete Applied Mathematics*, vol. 169, pp. 30-42, 2014.

[6] K. J. Batenburg, W. A. Kosters, “Solving Nonograms by Combining Relaxations,” *Pattern Recognition*,vol. 42, no. 8, pp. 1672-1683, 2009.

[7] S. J. Yen, T. C. Su, S. Y. Chiu, J. C.Chen, “Optimization of Nonogram’s Solver by Using an Efficient Algorithm”, *Proceedings of International Conference on Technologies and Applications of Artificial Intelligence (TAAI)*, 2010.

[8] M. Dorant, “Pic-a-Pix Tutorial and Walkthroughs: A beginner’s guide to solving picture logic puzzles,” September 12, 2005, https://www.conceptispuzzles.com/zh/index.aspx?uri=info/article/79

[9] 宋得榮、黃國展，”Nonograms 中單行求解方法探討”, TCGA 2020 電腦對局研討會, 東海大學, 2020 年 7 月 24, 25 日。

[10] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[11] J. Pearl, D. Mackenzie, *The Book of Why: The New Science of Cause and Effect*, Basic Books, 2018.

[12] J. Pearl, “The Seven Tools of Causal Inference, with Reflections on Machine Learning”, *Communications of the ACM*, Vol. 62, No. 3, pp. 54-60, March 2019.

[13] A. T. H. Sim, M. Indrawan, S. Zutshi, B. Srinivasan, “Logic-Based Pattern Discovery”, *IEEE Transactions on Knowledge and Data Engineering*, Vol. 22, pp. 798-811, June 2010.

[14] P. K. Novak, N. Lavraˇ, G. I. Webb, “Supervised Descriptive Rule Discovery: A Unifying Survey of Contrast Set, Emerging Pattern and Subgroup Mining”, Journal of Machine Learning Research, Vol. 10, pp. 377-403, 2009.

[15] J. Li, “On Optimal Rule Discovery”, *IEEE Transactions on Knowledge and Data Engineering*, Vol. 18, Iss. 4, pp. 460-471, April 2006.

[16] X. Shen, S. Ma, P. Vemuri, G. Simon, “Challenges and Opportunities with Causal Discovery Algorithms: Application to Alzheimer’s Pathophysiology”, *Scientific Reports*, 10, 2975, 2020.

[17] C. Glymour, K. Zhang, P. Spirtes, “Review of Causal Discovery Methods Based on Graphical Models”, Frontiers in Genetics, June 2019.

[18] F. Eberhardt, “Introduction to the Foundations of Causal Discovery”, International Journal of Data Science and Analytics, Vol. 3, pp. 81–91, 2017.

[19] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, P. S. Yu, “A Comprehensive Survey on Graph Neural Networks”, *IEEE Transactions on Neural Networks and Learning Systems*, pp. 1-21, March 2020.

[20] 葉家郡, 黃國展, “Nonogram 解題加速方法探討,” TCGA 2015 電腦對局研討會, 中原大學, 2015 年 5 月 29, 30 日。